

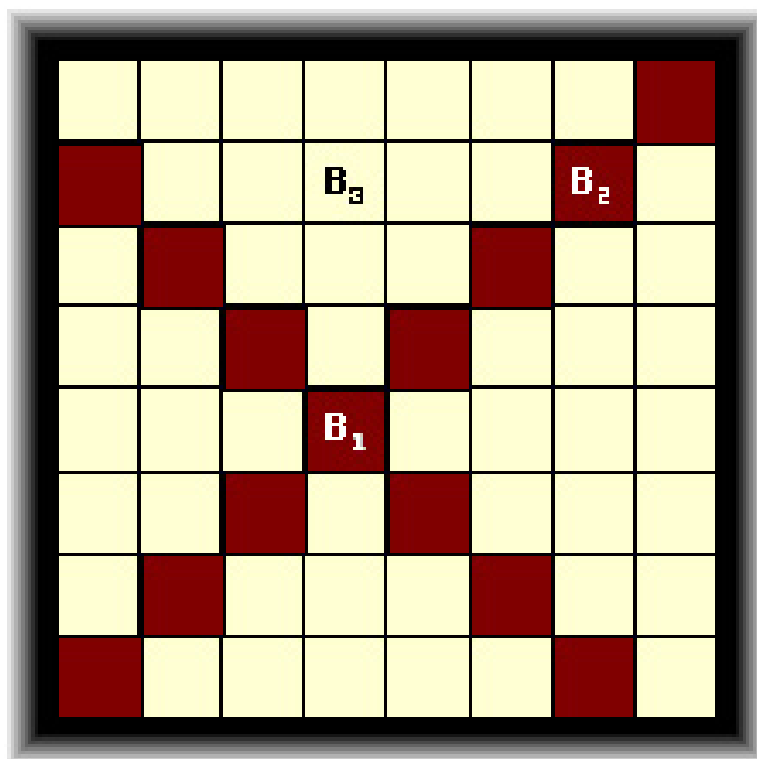
UVa10237 Bishops

题目链接

[UVa10237 Bishops](#)

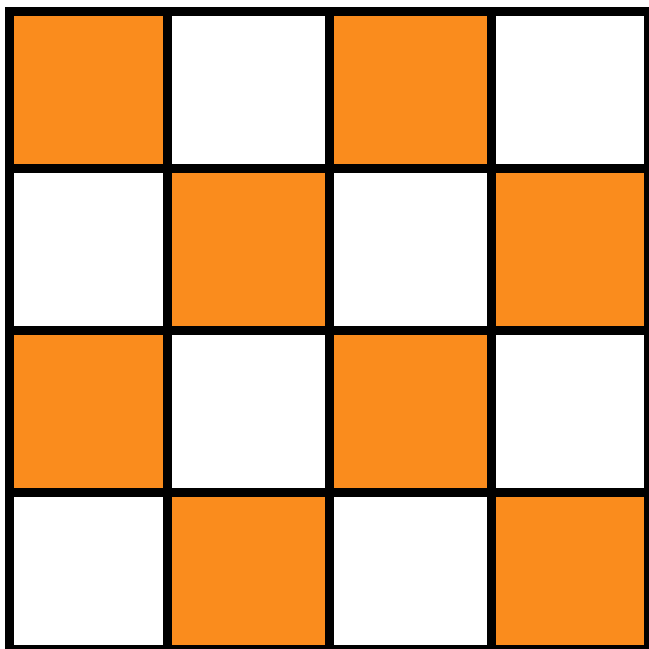
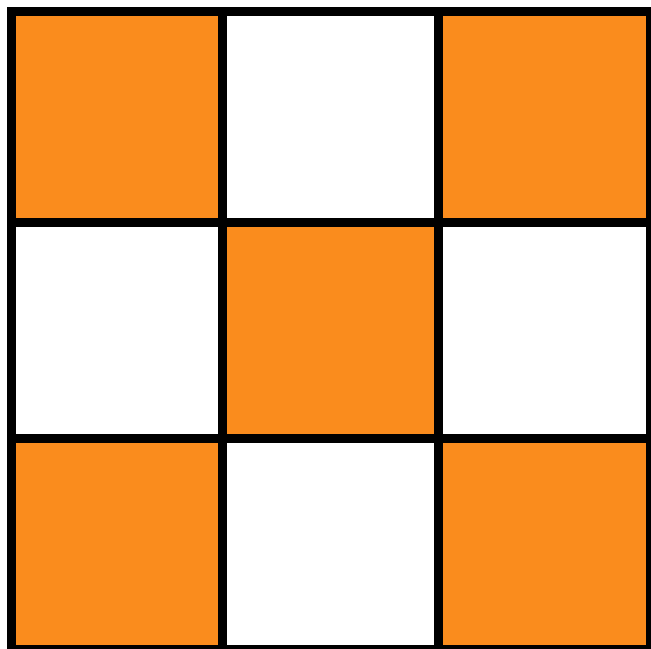
题意

两个象互不攻击，当且仅当它们不处于同一条斜线上。输入整数 n ($n \leq 30$)，统计在一个 $n \times n$ 的棋盘上放 k 个互不攻击的象有多少种方法。如 $n=8$, $k=6$ 时有 5 599 888 种。

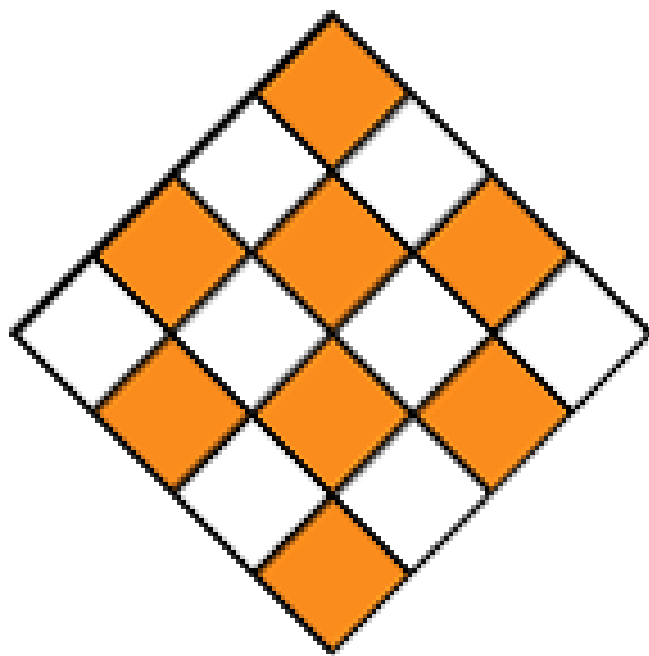
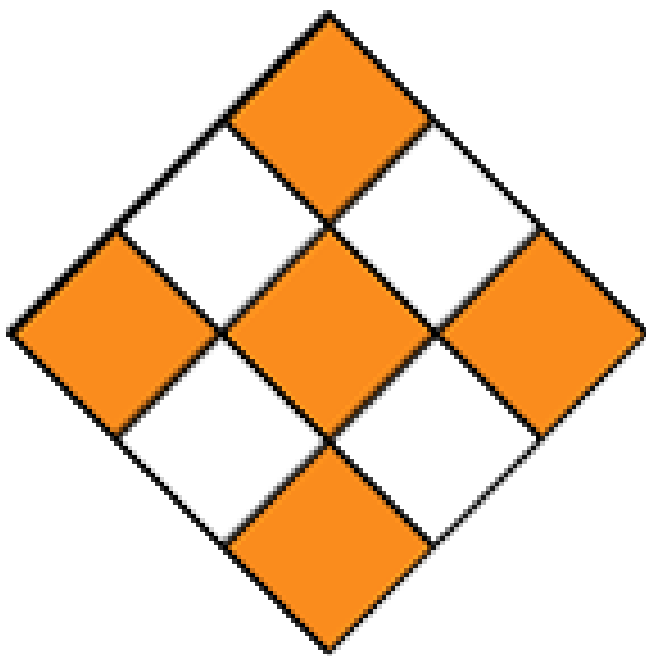


分析

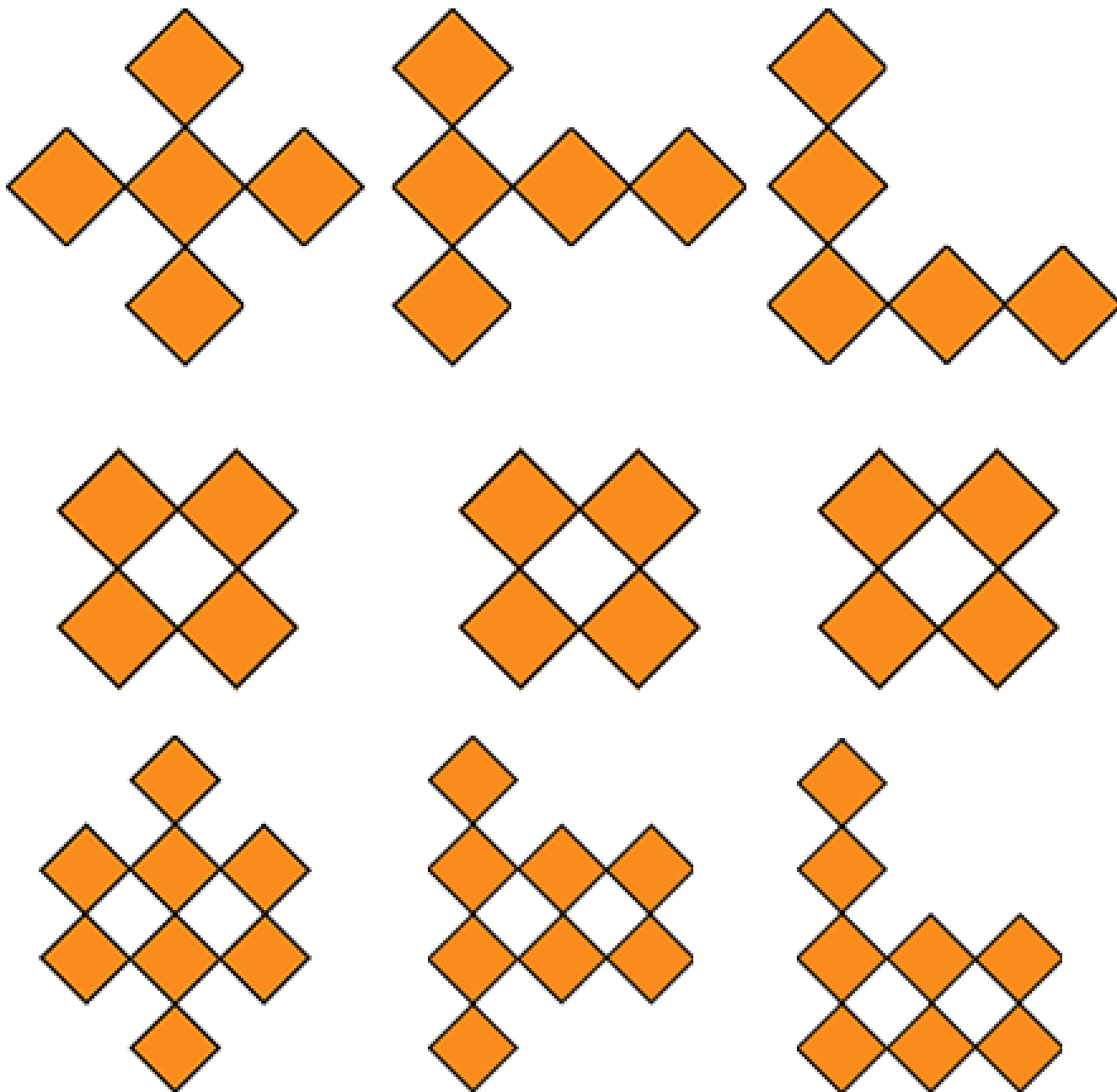
棋盘上能相互攻击的方格可以分成两类，两类之间任意方格上的象无法相互攻击，可以独立考虑。



还可以看出， n 为偶数时两分类不相同， n 为奇数时两分类相同。每个分类所含的方格如果放象，则可能相互攻击，把每个分类对应的图旋转 45° 变正，则每条水平 / 竖直线上最多有一个格子放象。



任意列可以交换顺序，计数的结果不变，因此可以将每种分类按列的长度从大到小重排，重新排后，任意行也可以交换顺序，最后可以变成阶梯形状。



经过这样的一个脑洞过程，可以定义一个多维递推状态： $f[h_1][c_1][h_2][c_2][k]$ ($f[N][2][N][2][N]$)，表示阶梯最大高度为 h_1 ，最大高度的列有 c_1+1 个 ($c_1 < 2$)，最小高度为 h_2 ，最小高度的列有 c_2 个 ($c_2 < 2$) 放 k 各象时部相互攻击的摆放方法计数。

状态转移就很清晰了：高度最小的列上可以不放象，计数为 $f[h_1][c_1][h_2][c_2-1][k]$ ；高度最小的列上放一个象，计数为和 $h_2 * f[h_1-1][c_1][h_2-1][c_2-1][k]$ 。

递推式为 $f[h_1][c_1][h_2][c_2][k] = f[h_1][c_1][h_2][c_2-1][k] + h_2 * f[h_1-1][c_1][h_2-1][c_2-1][k]$ 。

考虑一些实际的边界情况，状态转移会再复杂一点，但主体思路就是这样，时间复杂度为 $O(n^3)$

很烧脑的一道题目，本人很享受脑洞出多维递推状态的过程，更多细节参见 AC 代码。

AC 代码

```
1  #include <iostream>
2  using namespace std;
3
4  #define N 31
5  long long f[N][2][N][2][N] = {0}; int n, k;
6
7  long long solve() {
8      if (n == 1) return k<=1;
9      if (k > 2*n-2) return 0;
10     if (n & 1) {
11         long long (&r1)[N] = f[n][0][1][1], (&r2)[N] = f[n-1][1][n>3 ? 2 : 0][1
12     ], ans = 0;
13         for (int i=0; i<=k && i<n; ++i) ans += r1[i]*r2[k-i];
14         return ans;
15     }
16     long long (&r)[N] = f[n][0][n>2 ? 2 : 0][1], ans = 0;
17     for (int i=0; i<=k && i<n; ++i) ans += r[i]*r[k-i];
18     return ans;
19 }
20 int main() {
21     for (int h1=2; h1<N; ++h1) for (int c1=0; c1<2; ++c1) {
22         f[h1][c1][0][0][0] = f[h1][c1][0][1][0] = 1;
23         f[h1][c1][0][0][1] = f[h1][c1][0][1][1] = (c1+1)*h1;
24         if (c1) f[h1][c1][0][0][2] = f[h1][c1][0][1][2] = h1*(h1-1);
25         for (int h2 = h1-2; h2>0; h2 -= 2) for (int c2=0; c2<2; ++c2) {
26             f[h1][c1][h2][c2][0] = 1;
27             for (int k=1, m=min(h1, h1-h2+c1+c2); k<=m; ++k) {
28                 f[h1][c1][h2][c2][k] = f[h1][c1][c2 ? h2 : (h2+2<h1 ? h2+2 : 0)][!c
29                 2][k]
30                     + h2 * f[h1-1][c1][c2 ? (h2>1 ? h2-1 : (h1>3 ? 2 : 0)) :
31                     (h2+2<h1 ? h2+1 : 0)][h2>1 ? !c2 : 1][k-1];
32             }
33         }
34     }
35     while (cin>>n>>k && n) cout << solve() << endl;
36     return 0;
37 }
```